

4

DECIDIBILIDADE

No Capítulo 3, introduzimos a máquina de Turing como um modelo de um computador de propósito geral e definimos a noção de algoritmo em termos de máquinas de Turing por meio da tese de Church-Turing.

Neste capítulo, começamos a investigar o poder de algoritmos para resolver problemas. Exibimos certos problemas que podem ser resolvidos algorítmicamente e outros que não podem. Nosso objetivo é explorar os limites da solubilidade algorítmica. Você está provavelmente familiarizado com solubilidade por algoritmos, pois muito da ciência da computação é dedicado a resolver problemas. A insolubilidade de certos problemas pode vir como uma surpresa.

Por que você deveria estudar insolubilidade? Afinal de contas, mostrar que um problema é insolúvel não parece ser de nenhuma utilidade se você tem de resolvê-lo. Você precisa estudar esse fenômeno por duas razões. Primeiro, saber quando um problema é algorítmicamente insolúvel é útil porque então você se dá conta de que o problema tem que ser simplificado ou alterado antes que possa encontrar uma solução algorítmica. Como qualquer ferramenta, os computadores têm capacidades e limitações que devem ser reconhecidas caso se queira que eles sejam bem usados. A segunda razão é cultural. Mesmo se você lidar com problemas que claramente são solúveis, um vislumbre do insolúvel pode estimular sua imaginação e ajudá-lo a ganhar uma perspectiva importante sobre computação.

IDÉIA DA PROVA Simplesmente precisamos apresentar uma MT M que decide A_{AFD} .

$M =$ “Sobre a entrada $\langle B, w \rangle$, onde B é um AFD, e w , uma cadeia:

1. Simule B sobre a entrada w .
2. Se a simulação termina em um estado de aceitação, *aceite*. Se ela termina em um estado de não-aceitação, *rejeite*.”

PROVA Mencionamos apenas alguns poucos detalhes de implementação dessa prova. Para aqueles familiarizados em escrever programas em alguma linguagem de programação convencional, imagine como você escreveria um programa para realizar a simulação.

Primeiro, vamos examinar a entrada $\langle B, w \rangle$. Ela é uma representação de um AFD B juntamente com uma cadeia w . Uma representação razoável de B é simplesmente uma lista de seus cinco componentes, Q, Σ, δ, q_0 e F . Quando M recebe sua entrada, M primeiro determina se ela representa apropriadamente um AFD B e uma cadeia w . Se não, M rejeita.

Então, M realiza a simulação diretamente. Ela mantém registro do estado atual de B e da posição atual de B na entrada w escrevendo essa informação na sua fita. Inicialmente, o estado atual de B é q_0 e a posição atual de B sobre a entrada é o símbolo mais à esquerda de w . Os estados e a posição são atualizados conforme a função de transição especificada δ . Quando M termina de processar o último símbolo de w , M aceita a entrada se B estiver em um estado de aceitação; M rejeita a entrada se B estiver em um estado de não-aceitação.

Podemos provar um teorema similar para autômatos finitos não-determinísticos. Seja

$$A_{AFN} = \{ \langle B, w \rangle \mid B \text{ é um AFN que aceita a cadeia de entrada } w \}.$$

TEOREMA 4.2

A_{AFN} é uma linguagem decidível.

PROVA Apresentamos uma MT N que decide A_{AFN} . Poderíamos projetar N para operar como M , simulando um AFN em vez de um AFD. Ao invés disso, faremos diferentemente para ilustrar uma nova idéia: fazemos N usar M como uma sub-rotina. Como M foi projetada para funcionar com AFDs, N primeiro converte o AFN que ela recebe como entrada para um AFD antes de passá-lo para M .

$N =$ “Sobre a entrada $\langle B, w \rangle$ onde B é um AFN, e w , uma cadeia:

1. Converta AFN B para um AFD equivalente C , usando o procedimento para essa conversão dado no Teorema 1.39.

2. Rode a MT M do Teorema 4.1 sobre a entrada $\langle C, w \rangle$.
3. Se M aceita, *aceite*; caso contrário, *rejeite*.”

Rodar a MT M no estágio 2 significa incorporar M no projeto de N como um subprocedimento.

Similarmente, podemos determinar se uma expressão regular gera uma dada cadeia. Seja $A_{\text{EXR}} = \{\langle R, w \rangle \mid R \text{ é uma expressão regular que gera a cadeia } w\}$.

TEOREMA 4.3

A_{EXR} é uma linguagem decidível.

PROVA A seguinte MT P decide A_{EXR} .

$P =$ “Sobre a entrada $\langle R, w \rangle$ onde R é uma expressão regular e w é uma cadeia:

1. Converta a expressão regular R para um AFN equivalente A usando o procedimento para essa conversão dado no Teorema 1.54.
2. Rode a MT N sobre a entrada $\langle A, w \rangle$.
3. Se N aceita, *aceite*; se N rejeita, *rejeite*.”

Os Teoremas 4.1, 4.2 e 4.3 ilustram que, para os propósitos de decidibilidade, entregar à máquina de Turing um AFD, um AFN ou uma expressão regular, é tudo equivalente, pois a máquina é capaz de converter uma forma de codificação na outra.

Agora nos voltamos para um tipo diferente de problema concernente a autômatos finitos: *testar vacuidade* para a linguagem de um autômato finito. Nos três teoremas precedentes tivemos de determinar se um autômato finito aceita uma cadeia específica. Na próxima prova temos que determinar se um autômato finito aceita alguma cadeia. Seja

$$V_{\text{AFD}} = \{\langle A \rangle \mid A \text{ é um AFD e } L(A) = \emptyset\}.$$

TEOREMA 4.4

V_{AFD} é uma linguagem decidível.

PROVA Um AFD aceita alguma cadeia sse é possível atingir um estado de aceitação a partir do estado inicial passando pelas setas do AFD. Para testar essa

condição, podemos projetar uma MT T que usa um algoritmo de marcação similar àquele utilizado no Exemplo 3.23.

T = “Sobre a entrada $\langle A \rangle$ onde A é um AFD:

1. Marque o estado inicial de A .
2. Repita até que nenhum estado novo venha a ser marcado:
3. Marque qualquer estado que tenha uma transição chegando nele a partir de qualquer estado que já está marcado.
4. Se nenhum estado de aceitação estiver marcado, *aceite*; caso contrário, *rejeite*.”

O próximo teorema afirma que determinar se dois AFDs reconhecem a mesma linguagem é decidível. Seja

$$EQ_{AFD} = \{ \langle A, B \rangle \mid A \text{ e } B \text{ são AFDs e } L(A) = L(B) \}.$$

TEOREMA 4.5

EQ_{AFD} é uma linguagem decidível.

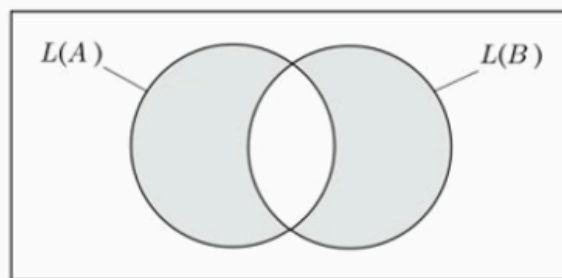
PROVA Para provar esse teorema, usamos o Teorema 4.4. Construímos um novo AFD C a partir de A e B , tal que C aceita somente aquelas cadeias que são aceitas ou por A ou por B , mas não por ambos. Consequentemente, se A e B reconhecem a mesma linguagem, C não aceitará nada. A linguagem de C é

$$L(C) = (L(A) \cap \overline{L(B)}) \cup (\overline{L(A)} \cap L(B)).$$

Essa expressão é, às vezes, chamada *diferença simétrica* de $L(A)$ e $L(B)$ e é ilustrada na Figura 4.6. Aqui $\overline{L(A)}$ é complemento de $L(A)$. A diferença simétrica é útil aqui porque $L(C) = \emptyset$ sse $L(A) = L(B)$. Podemos obter C a partir de A e B com as construções utilizadas para provar que a classe das linguagens regulares é fechada sob complementação, união e interseção. Essas construções são algoritmos que podem ser realizados por máquinas de Turing. Uma vez tendo construído C , podemos usar o Teorema 4.4 para testar se $L(C)$ é vazia. Se ela for vazia, $L(A)$ e $L(B)$ têm de ser iguais.

F = “Sobre a entrada $\langle A, B \rangle$, onde A e B são AFDs:

1. Construa o AFD C conforme descrito.
2. Rode a MT T do Teorema 4.4 sobre a entrada $\langle C \rangle$.
3. Se T aceita, *aceite*. Se T rejeita, *rejeite*.”

**FIGURA 4.6**

A diferença simétrica de $L(A)$ e $L(B)$.

PROBLEMAS DECIDÍVEIS CONCERNENTES A LINGUAGENS LIVRES-DO-CONTEXTO

Aqui, descrevemos algoritmos para determinar se uma GLC gera uma cadeia específica e para determinar se a linguagem de um GLC é vazia. Seja

$$A_{\text{GLC}} = \{ \langle G, w \rangle \mid G \text{ é uma GLC que gera a cadeia } w \}.$$

TEOREMA 4.7

A_{GLC} é uma linguagem decidível.

IDÉIA DA PROVA Para a GLC G e a cadeia w , queremos determinar se G gera w . Uma idéia é usar G para passar por todas as derivações para determinar se alguma delas é uma derivação de w . Essa idéia não funciona, pois uma quantidade infinita de derivações pode ter que ser testada. Se G não gera w , esse algoritmo nunca pararia. Essa idéia leva a uma máquina de Turing que é um reconhecedor, mas não um decisor, para A_{GLC} .

Para tornar essa máquina de Turing um decisor, precisamos garantir que o algoritmo tenta somente uma quantidade finita de derivações. No Problema 2.26 (página 136), mostramos que, se G estivesse na forma normal de Chomsky, qualquer derivação de w teria $2n - 1$ passos, onde n é o comprimento de w . Nesse caso, verificar apenas as derivações com $2n - 1$ passos para determinar se G gera w seria suficiente. Existe somente uma quantidade finita de tais derivações. Podemos converter G para a forma normal de Chomsky, usando o procedimento dado na Seção 2.1.

PROVA A MT S para A_{GLC} segue.

S = “Sobre a entrada $\langle G, w \rangle$, onde G é uma GLC, e w , uma cadeia:

1. Converta G para uma gramática equivalente na forma normal de Chomsky.
2. Liste todas as derivações com $2n - 1$ passos, onde n é o comprimento de w , exceto se $n = 0$; nesse último caso, liste todas as derivações com 1 passo.
3. Se alguma dessas derivações gera w , *aceite*; se não, *rejeite*.”

O problema de se determinar se uma GLC gera uma cadeia específica está relacionado ao problema de compilar linguagens de programação. O algoritmo na MT S é muito ineficiente e nunca seria utilizado na prática, mas é fácil de descrever e não estamos preocupados com eficiência aqui. Na Parte Três deste livro, abordamos questões concernentes ao tempo de execução e ao uso de memória de algoritmos. Na prova do Teorema 7.16, descrevemos um algoritmo mais eficiente para reconhecer linguagens livres-do-contexto.

Lembre-se de que demos procedimentos para converter ida e volta entre GLCs e APs no Teorema 2.20. Logo, tudo que dizemos sobre a decidibilidade de problemas concernentes a GLCs se aplica igualmente bem a APs.

Vamos nos voltar agora para o problema de se testar vacuidade para a linguagem de uma GLC. Como fizemos para os AFDs, podemos mostrar que o problema de se determinar se uma GLC gera alguma cadeia é decidível. Seja

$$V_{GLC} = \{ \langle G \rangle \mid G \text{ é uma GLC e } L(G) = \emptyset \}.$$

TEOREMA 4.8

V_{GLC} é uma linguagem decidível.

IDÉIA DA PROVA Para encontrar um algoritmo para esse problema, poderíamos tentar usar a MT S do Teorema 4.7. Ele afirma que podemos testar se uma GLC gera alguma cadeia específica w . Para determinar se $L(G) = \emptyset$, o algoritmo poderia tentar passar por todas as possíveis ws , uma por uma. Mas existe uma quantidade infinita de ws para se tentar; logo, esse método poderia terminar rodando para sempre. Precisamos tomar uma abordagem diferente.

Para determinar se a linguagem de uma gramática é vazia, precisamos testar se a variável inicial pode gerar uma cadeia de terminais. O algoritmo faz isso resolvendo um problema mais geral. Ele determina, *para cada variável*, se ela é capaz de gerar uma cadeia de terminais. Quando o algoritmo tiver determinado

que uma variável pode gerar alguma cadeia de terminais, ele mantém registro dessa informação, colocando uma marca sobre essa variável.

Primeiro, o algoritmo marca todos os símbolos terminais na gramática. Então, ele faz uma varredura em todas as regras da gramática. Se por acaso encontrar uma regra que permite alguma variável ser substituída por alguma cadeia de símbolos dos quais todos já estejam marcados, o algoritmo sabe que essa variável pode ser marcada, também. O algoritmo continua dessa forma até que não possa marcar mais nenhuma variável. A MT R implementa esse algoritmo.

PROVA

$R =$ “Sobre a entrada $\langle G \rangle$, onde G é uma GLC:

1. Marque todos os símbolos terminais em G .
2. Repita até que nenhuma variável venha a ser marcada:
3. Marque qualquer variável A onde G tem uma regra $A \rightarrow U_1 U_2 \cdots U_k$ e cada símbolo $U_1, \dots, U_k$ já tenha sido marcado.
4. Se a variável inicial não está marcada, *aceite*; caso contrário, *rejeite*.”

A seguir, consideramos o problema de se determinar se duas gramáticas livres-do-contexto geram a mesma linguagem. Seja

$$EQ_{GLC} = \{ \langle G, H \rangle \mid G \text{ e } H \text{ são GLCs e } L(G) = L(H) \}.$$

O Teorema 4.5 deu um algoritmo que decide a linguagem análoga EQ_{AFD} para autômatos finitos. Usamos o procedimento de decisão para V_{AFD} para provar que EQ_{AFD} é decidível. Como V_{GLC} também é decidível, você poderia então pensar que podemos usar uma estratégia similar para provar que EQ_{GLC} é decidível. Mas algo está errado com essa idéia! A classe de linguagens livres-do-contexto *não* é fechada sob complementação ou interseção, como você provou no Exercício 2.2. Na verdade, EQ_{GLC} não é decidível. A técnica para provar isso é apresentada no Capítulo 5.

Agora mostramos que toda linguagem livre-do-contexto é decidível por uma máquina de Turing.

TEOREMA 4.9

Toda linguagem livre-do-contexto é decidível.

IDÉIA DA PROVA Seja A uma LLC. Nosso objetivo é mostrar que A é decidível. Uma (má) idéia é converter um AP para A diretamente em uma MT. Isso

não é difícil de se fazer, porque simular uma pilha com a fita mais versátil de uma MT é fácil. O AP para A pode ser não-determinístico, mas isso parece não acarretar problemas, pois podemos convertê-lo em uma MT não-determinística e sabemos que qualquer MT não-determinística pode ser convertida em uma MT determinística equivalente. Ainda assim, existe uma dificuldade. Alguns ramos da computação do AP podem rodar para sempre, lendo e escrevendo na pilha sem nunca parar. A MT simuladora então também teria ramos não-terminantes em sua computação e, portanto, a MT não seria um decisor. Uma idéia diferente é necessária. Ao invés, provamos esse teorema com a MT S que projetamos no Teorema 4.7 para decidir A_{GLC} .

PROVA Seja G uma GLC para A e projetemos uma MT M_G que decide A . Construímos uma cópia de G dentro de M_G . Ela funciona da seguinte maneira.

$M_G =$ "Sobre a entrada w :

1. Rode a MT S sobre a entrada $\langle G, w \rangle$.
2. Se essa máquina aceita, *aceite*; se ela rejeita, *rejeite*."

O Teorema 4.9 provê a ligação final no relacionamento entre as quatro principais classes de linguagens que descrevemos até agora: regulares, livres-do-contexto, decidíveis e Turing-reconhecíveis. A Figura 4.10 mostra esse relacionamento.

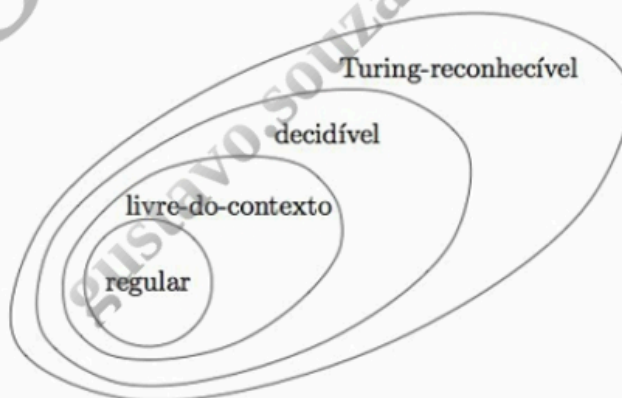


FIGURA 4.10
O relacionamento entre classes de linguagens.

4.2

O PROBLEMA DA PARADA

Nesta seção provamos um dos teoremas mais importantes filosoficamente da teoria da computação: existe um problema específico que é algoritmicamente insolúvel. Os computadores parecem ser tão poderosos que você acredita que todos os problemas em algum momento cederão a eles. O teorema apresentado aqui demonstra que computadores são limitados de uma maneira fundamental.

Que tipos de problemas são insolúveis por computador? São problemas esótericos, residindo somente nas mentes dos teóricos? Não! Mesmo alguns problemas comuns que as pessoas desejam resolver acontecem de ser computacionalmente insolúveis.

Em um tipo de problema insolúvel, você recebe um programa de computador e uma especificação precisa do que aquele programa supostamente faz (por exemplo, ordenar uma lista de números). Você precisa verificar que o programa funciona conforme o especificado (isto é, que ele é correto). Como tanto o programa quanto a especificação são objetos matematicamente precisos, você espera automatizar o processo de verificação alimentando esses objetos a um computador apropriadamente programado. Entretanto, você ficará desapontado. O problema geral de verificação de software não é solúvel por computador.

Nesta seção e no Capítulo 5 você encontrará diversos problemas computacionalmente insolúveis. Nossos objetivos são ajudá-lo a desenvolver um sentimento dos tipos de problemas que são insolúveis e a aprender técnicas para provar a insolubilidade.

Agora nos voltamos para nosso primeiro teorema que estabelece a indecidibilidade de uma linguagem específica: o problema de se determinar se uma máquina de Turing aceita uma dada cadeia de entrada. Chamamo-lo A_{MT} por analogia com A_{AFD} e A_{GLC} . Mas, enquanto A_{AFD} e A_{GLC} eram decidíveis, A_{MT} não o é. Seja

$$A_{MT} = \{ \langle M, w \rangle \mid M \text{ é uma MT e } M \text{ aceita } w \}.$$

TEOREMA 4.11

A_{MT} é indecidível.

Antes de chegar na prova, vamos primeiro observar que A_{MT} é Turing-reconhecível. Portanto, esse teorema afirma que reconhecedores *são* mais poderosos que decisores. Requerer que uma MT pare sobre todas as entradas restringe os tipos de linguagens que ela pode reconhecer. A máquina de Turing U a seguir reconhece A_{MT} .

U = “Sobre a entrada $\langle M, w \rangle$, onde M é uma MT e w é uma cadeia:

1. Simule M sobre a entrada w .
2. Se M em algum momento entra no seu estado de aceitação,

aceite; se M em algum momento entra em seu estado de rejeição, *rejeite*.”

Note que essa máquina entra em *loop* sobre a entrada $\langle M, w \rangle$ se M entra em *loop* sobre w , e é por isso que essa máquina não decide A_{MT} . Se o algoritmo tivesse alguma forma de determinar que M não iria parar sobre w , ele poderia dizer *rejeite*. Logo, A_{MT} é, às vezes, denominado **problema da parada**. Como vamos demonstrar, um algoritmo não tem como fazer essa determinação.

A máquina de Turing U é interessante em si mesma. Ela é um exemplo da *máquina de Turing universal* primeiro proposta por Turing. Essa máquina é chamada de universal porque é capaz de simular qualquer outra máquina de Turing a partir da descrição da mesma. A máquina de Turing universal desempenhou um importante papel inicial no estímulo ao desenvolvimento de computadores de programa-armazenado.

O MÉTODO DA DIAGONALIZAÇÃO

A prova da indecidibilidade do problema da parada usa uma técnica chamada *diagonalização*, descoberta pelo matemático Georg Cantor em 1873. Cantor estava preocupado com o problema de se medir os tamanhos de conjuntos infinitos. Se tivermos dois conjuntos infinitos, como podemos dizer se um é maior que o outro ou se eles têm o mesmo tamanho? Para os conjuntos finitos, é claro, responder a essas questões é fácil. Simplesmente contamos os elementos em um conjunto finito e o número resultante é seu tamanho. Mas, se tentarmos contar os elementos de um conjunto infinito, nunca terminaremos! Portanto, não podemos usar o método de contagem para determinar os tamanhos relativos de conjuntos infinitos.

Por exemplo, tome o conjunto dos inteiros pares e o conjunto de todas as cadeias sobre $\{0,1\}$. Ambos os conjuntos são infinitos e, portanto, maiores que qualquer conjunto finito, mas um dos dois é maior que o outro? Como podemos comparar seu tamanho relativo?

Cantor propôs uma solução elegante para esse problema. Ele observou que dois conjuntos finitos têm o mesmo tamanho se os elementos de um deles puderem ser emparelhados com os elementos do outro. Esse método compara os tamanhos sem recorrer a contagem. Podemos estender essa idéia para conjuntos infinitos. Vamos ver o que isso significa mais precisamente.

DEFINIÇÃO 4.12

Suponha que tenhamos os conjuntos A e B e uma função f de A para B . Digamos que f é **um-para-um** se ela nunca mapeia dois elementos diferentes para um mesmo lugar — ou seja, se $f(a) \neq f(b)$ sempre que $a \neq b$. Digamos que f é **sobrejetora** se ela atinge todo elemento de B — ou seja, se para todo $b \in B$ existe um $a \in A$ tal que $f(a) = b$. Digamos que A e B são de **mesmo tamanho** se existe uma função um-para-um e sobrejetora $f: A \rightarrow B$. Uma função que é tanto um-para-um quanto sobrejetora é denominada uma **correspondência**. Em uma correspondência, todo elemento de A mapeia para um único elemento de B e cada elemento de B tem um único elemento de A mapeando para ele. Uma correspondência é simplesmente uma maneira de emparelhar os elementos de A com os elementos de B .

EXEMPLO 4.13

Seja $\mathcal{N}$ o conjunto de números naturais $\{1, 2, 3, \dots\}$ e suponha que $\mathcal{E}$ seja o conjunto dos números naturais pares $\{2, 4, 6, \dots\}$. Usando a definição de Cantor de tamanho podemos ver que $\mathcal{N}$ e $\mathcal{E}$ têm o mesmo tamanho. A correspondência f mapeando $\mathcal{N}$ para $\mathcal{E}$ é simplesmente $f(n) = 2n$. Podemos visualizar f mais facilmente com a ajuda de uma tabela.

n	$f(n)$
1	2
2	4
3	6
$\vdots$	$\vdots$

É claro que esse exemplo parece bizarro. Intuitivamente, $\mathcal{E}$ parece menor que $\mathcal{N}$ porque $\mathcal{E}$ é um subconjunto próprio de $\mathcal{N}$. Mas emparelhar cada membro de $\mathcal{N}$ com seu próprio membro de $\mathcal{E}$ é possível, portanto declaramos esses dois conjuntos como sendo de mesmo tamanho. ■

DEFINIÇÃO 4.14

Um conjunto A é **contável** se é finito ou tem o mesmo tamanho que $\mathcal{N}$.

EXEMPLO 4.15

Agora nos voltamos para um exemplo ainda mais estranho. Se $\mathcal{Q} = \{\frac{m}{n} \mid m, n \in \mathcal{N}\}$ for o conjunto de números racionais positivos, $\mathcal{Q}$ parece ser muito maior que $\mathcal{N}$. Ainda assim, esses dois conjuntos são do mesmo tamanho conforme

nossa definição. Damos uma correspondência com $\mathcal{N}$ para mostrar que $\mathcal{Q}$ é contável. Uma maneira fácil de fazer isso é listar todos os elementos de $\mathcal{Q}$. Então emparelhamos o primeiro elemento na lista com o número 1 de $\mathcal{N}$, o segundo elemento na lista com o número 2 de $\mathcal{N}$, e assim por diante. Temos de garantir que todo membro de $\mathcal{Q}$ aparece somente uma vez na lista.

Para obter essa lista fazemos uma matriz infinita contendo todos os números racionais positivos, como mostrado na Figura 4.16. A i -ésima linha contém todos os números com numerador i e a j -ésima coluna tem todos os números com denominador j . Portanto, o número $\frac{i}{j}$ ocorre na i -ésima linha e j -ésima coluna.

Agora transformamos essa matriz em uma lista. Uma forma (ruim) de se tentar isso seria começar a lista com todos os elementos na primeira linha. Essa não é uma boa abordagem porque a primeira linha é infinita, portanto a lista nunca chegaria à segunda linha. Em vez disso, listamos os elementos nas diagonais, começando do canto, que estão sobrepostos no diagrama. A primeira diagonal contém o único elemento $\frac{1}{1}$, e a segunda diagonal contém os dois elementos $\frac{2}{1}$ e $\frac{1}{2}$. Logo, os primeiros três elementos na lista são $\frac{1}{1}$, $\frac{2}{1}$ e $\frac{1}{2}$. Na terceira diagonal uma complicação aparece. Ela contém $\frac{3}{1}$, $\frac{2}{2}$ e $\frac{1}{3}$. Se simplesmente adicionássemos esses à lista, repetiríamos $\frac{1}{1} = \frac{2}{2}$. Evitamos fazer isso saltando um elemento quando ele causasse uma repetição. Dessa forma, adicionamos apenas os dois novos elementos, $\frac{3}{1}$ e $\frac{1}{3}$. Continuando dessa maneira, obtemos uma lista de todos os elementos de $\mathcal{Q}$.



FIGURA 4.16

Uma correspondência de $\mathcal{N}$ e $\mathcal{Q}$.

Depois de ver a correspondência de $\mathcal{N}$ e $\mathcal{Q}$, você pode achar que quaisquer dois conjuntos infinitos podem ser mostrados como tendo o mesmo tamanho. Afinal de contas, você só precisa exibir uma correspondência, e esse exemplo revela que correspondências surpreendentes realmente existem. Entretanto,

para alguns conjuntos infinitos, nenhuma correspondência com $\mathcal{N}$ existe. Esses conjuntos são simplesmente grandes demais. Tais conjuntos são chamados *incontáveis*.

O conjunto dos números reais é um exemplo de conjunto incontável. Um *número real* é aquele que tem uma representação decimal. Os números $\pi = 3,1415926\dots$ e $\sqrt{2} = 1,4142135\dots$ são exemplos de números reais. Seja $\mathcal{R}$ o conjunto dos números reais. Cantor provou que $\mathcal{R}$ é incontável. Ao fazer isso ele introduziu o método da diagonalização.

TEOREMA 4.17

$\mathcal{R}$ é incontável.

PROVA Para mostrar que $\mathcal{R}$ é incontável, mostramos que não existe correspondência entre $\mathcal{N}$ e $\mathcal{R}$. A prova é por contradição. Suponha que existisse uma correspondência f entre $\mathcal{N}$ e $\mathcal{R}$. Nossa tarefa é mostrar que f não funciona como deveria. Para ela ser uma correspondência, f deve emparelhar todos os membros de $\mathcal{N}$ com todos os membros de $\mathcal{R}$. Mas encontraremos um x em $\mathcal{R}$ que não é emparelhado com nada em $\mathcal{N}$, o que será nossa contradição.

A maneira pela qual encontramos esse x é verdadeiramente construindo-o. Escolhemos cada dígito de x para tornar x diferente de um dos números reais que está emparelhado com um elemento de $\mathcal{N}$. No final, asseguramos que x seja diferente de qualquer número real que esteja emparelhado.

Podemos ilustrar essa idéia dando um exemplo. Suponha que a correspondência f exista. Seja $f(1) = 3,14159\dots$, $f(2) = 55,55555\dots$, $f(3) = \dots$, e assim por diante, só para dar alguns valores para f . Então f emparelha o número 1 com $3,14159\dots$, o número 2 com $55,55555\dots$, e assim por diante. A tabela abaixo mostra uns poucos valores de uma correspondência hipotética f entre $\mathcal{N}$ e $\mathcal{R}$.

n	$f(n)$
1	3,14159...
2	55,55555...
3	0,12345...
4	0,50000...
$\vdots$	$\vdots$

Construímos o x desejado dando sua representação decimal. Trata-se de um número entre 0 e 1, portanto, todos os seus dígitos significativos são dígitos fracionários posteriores à vírgula decimal. Nosso objetivo é assegurar que $x \neq f(n)$ para qualquer n . Para assegurar que $x \neq f(1)$, fazemos com que o primeiro dígito de x seja qualquer um diferente do primeiro dígito fracionário 1 de $f(1) = 3,14159\dots$. Arbitrariamente, fazemos com que ele seja 4. Para assegurar que $x \neq f(2)$ fazemos com que o segundo dígito de x seja qualquer coisa diferente do segundo dígito fracionário 5 de $f(2) = 55,55555\dots$. Arbitrariamente,

fazemos com que ele seja 6. O terceiro dígito fracionário de $f(3) = 0,12345\dots$ é 3, portanto fazemos com que x seja qualquer coisa diferente — digamos, 4. Continuando dessa maneira, ao longo da diagonal da tabela para f , obtemos todos os dígitos de x , como mostrado na tabela a seguir. Sabemos que x não é $f(n)$ para nenhum n , pois ele difere de $f(n)$ no n -ésimo dígito fracionário. (Um pequeno problema surge devido ao fato de que certos números, tais como $0,1999\dots$ e $0,2000\dots$, são iguais, muito embora suas representações decimais sejam diferentes. Evitamos esse problema nunca selecionando os dígitos 0 ou 9 quando construímos x .)

n	$f(n)$	
1	3, <u>1</u> 4159...	$x = 0,4641\dots$
2	55,5 <u>5</u> 555...	
3	0,1234 <u>5</u> ...	
4	0,500 <u>0</u> 0...	
$\vdots$	$\vdots$	

O teorema precedente tem uma aplicação importante para a teoria da computação. Ele mostra que algumas linguagens não são decidíveis ou mesmo Turing-reconhecíveis, pela razão de que existe uma quantidade incontável de linguagens e somente uma quantidade contável de máquinas de Turing. Visto que cada máquina de Turing pode reconhecer uma única linguagem e que existem mais linguagens que máquinas de Turing, algumas linguagens não são reconhecidas por nenhuma máquina de Turing. Tais linguagens não são Turing-reconhecíveis, como enunciamos no corolário seguinte.

COROLÁRIO 4.18

Algumas linguagens não são Turing-reconhecíveis.

PROVA Para mostrar que o conjunto de todas as máquinas de Turing é contável, primeiro observamos que o conjunto de todas as cadeias Σ^* é contável, para qualquer alfabeto Σ . Com apenas uma quantidade finita de cadeias de cada comprimento, podemos formar uma lista de Σ^* listando todas as cadeias comprimento 0, comprimento 1, comprimento 2, e assim por diante.

O conjunto de todas as máquinas de Turing é contável porque cada máquina de Turing M tem uma codificação em uma cadeia $\langle M \rangle$. Se simplesmente omitirmos aquelas cadeias que não são codificações legítimas de máquinas de Turing, podemos obter uma lista de todas as máquinas de Turing.

Para mostrar que o conjunto de todas as linguagens é incontável, primeiro observamos que o conjunto de todas as seqüências binárias infinitas é incontável. Uma *seqüência binária infinita* é uma seqüência interminável de 0s e 1s. Seja B o conjunto de todas as seqüências binárias infinitas. Podemos mostrar que B é incontável usando uma prova por diagonalização similar àquela que utilizamos no Teorema 4.17 para provar que $\mathcal{R}$ é incontável.

Seja $\mathcal{L}$ o conjunto de todas as linguagens sobre o alfabeto Σ . Mostramos que $\mathcal{L}$ é incontável dando uma correspondência com B , conseqüentemente mostrando que os dois conjuntos são do mesmo tamanho. Seja $\Sigma^* = \{s_1, s_2, s_3, \dots\}$. Cada linguagem $A \in \mathcal{L}$ tem uma seqüência única em B . O i -ésimo bit dessa seqüência é 1 se $s_i \in A$ e é 0 se $s_i \notin A$, o que é chamado de *seqüência característica* de A . Por exemplo, se A fosse a linguagem de todas as cadeias começando com 0 sobre o alfabeto $\{0,1\}$, sua seqüência característica χ_A seria

$$\begin{aligned}\Sigma^* &= \{ \varepsilon, 0, 1, 00, 01, 10, 11, 000, 001, \dots \}; \\ A &= \{ 0, 00, 01, 000, 001, \dots \}; \\ \chi_A &= 0 \quad 1 \quad 0 \quad 1 \quad 1 \quad 0 \quad 0 \quad 1 \quad 1 \quad \dots\end{aligned}$$

A função $f: \mathcal{L} \rightarrow B$, onde $f(A)$ é igual à seqüência característica de A , é um-para-um e sobrejetora e, portanto, uma correspondência. Conseqüentemente, como B é incontável, $\mathcal{L}$ também é incontável.

Assim, mostramos que o conjunto de todas as linguagens não pode ser posto em correspondência com o conjunto de todas as máquinas de Turing. Concluimos que algumas linguagens não são reconhecidas por nenhuma máquina de Turing.

O PROBLEMA DA PARADA É INDECIDÍVEL

Agora estamos prontos para provar o Teorema 4.11, a indecidibilidade da linguagem

$$A_{MT} = \{ \langle M, w \rangle \mid M \text{ é uma MT e } M \text{ aceita } w \}.$$

PROVA Supomos que A_{MT} seja decidível e obtemos uma contradição. Suponha que H seja um decisor para A_{MT} . Sobre a entrada $\langle M, w \rangle$, na qual M é uma MT, e w é uma cadeia, H pára e aceita se M aceita w . Além disso, H pára e rejeita se M falha em aceitar w . Em outras palavras, assumimos que H seja uma MT, onde

$$H(\langle M, w \rangle) = \begin{cases} \text{ aceite } & \text{ se } M \text{ aceita } w \\ \text{ rejeite } & \text{ se } M \text{ não aceita } w. \end{cases}$$

Agora construímos uma nova máquina de Turing D com H como uma sub-rotina. Essa nova MT chama H para determinar o que M faz quando a entrada para M é sua própria descrição $\langle M \rangle$. Uma vez que D tenha determinado essa informação, ela faz o oposto. Ou seja, ela rejeita se M aceita e aceita se M não aceita. O que segue é uma descrição de D .

$D =$ "Sobre a entrada $\langle M \rangle$, onde M é uma MT:

1. Rode H sobre a entrada $\langle M, \langle M \rangle \rangle$.
2. Dê como saída o oposto do que H dá como saída; ou seja, se H aceita, *rejeite* e se H rejeita, *aceite*."

Não se atrapalhe com a idéia de rodar uma máquina sobre sua própria descrição! Trata-se de rodar um programa consigo próprio como entrada, algo que de fato

ocorre ocasionalmente na prática. Por exemplo, um compilador é um programa que traduz outros programas. Um compilador para a linguagem Pascal pode, ele próprio, ser escrito em Pascal, portanto rodar esse programa sobre si próprio faria sentido. Em resumo,

$$D(\langle M \rangle) = \begin{cases} \text{ aceite } & \text{ se } M \text{ não aceita } \langle M \rangle \\ \text{ rejeite } & \text{ se } M \text{ aceita } \langle M \rangle. \end{cases}$$

O que acontece quando rodamos D com sua própria descrição $\langle D \rangle$ como entrada? Nesse caso, obtemos

$$D(\langle D \rangle) = \begin{cases} \text{ aceite } & \text{ se } D \text{ não aceita } \langle D \rangle \\ \text{ rejeite } & \text{ se } D \text{ aceita } \langle D \rangle. \end{cases}$$

Independentemente do que D faz, ela é forçada a fazer o oposto, o que é obviamente uma contradição. Conseqüentemente, nem a MT D nem a MT H podem existir.

Vamos revisar os passos dessa prova. Suponha que uma MT H decida A_{MT} . Então use H para construir uma MT D que, quando recebe uma dada entrada $\langle M \rangle$, aceita exatamente quando M não aceita a entrada $\langle M \rangle$. Finalmente, rode D sobre si própria. As máquinas tomam as seguintes ações, com a última linha sendo a contradição.

- H aceita $\langle M, w \rangle$ exatamente quando M aceita w .
- D rejeita $\langle M \rangle$ exatamente quando M aceita $\langle M \rangle$.
- D rejeita $\langle D \rangle$ exatamente quando D aceita $\langle D \rangle$.

Onde está a diagonalização na prova do Teorema 4.11? Ela se torna aparente quando você examina as tabelas de comportamento para as MTs H e D . Nessas tabelas listamos todas as MTs nas linhas, $M_1, M_2, \dots$ e todas as suas descrições nas colunas, $\langle M_1 \rangle, \langle M_2 \rangle, \dots$. As entradas dizem se a máquina em uma dada linha aceita a entrada em uma dada coluna. A entrada é *aceite* se a máquina aceita a entrada, mas é branco se ela rejeita ou entra em *loop* sobre aquela entrada. Montamos as entradas na Figura 4.19 para ilustrar a idéia.

Na Figura 4.20, as entradas são os resultados de se rodar H sobre as entradas correspondentes da Figura 4.19. Portanto, se M_3 não aceita a entrada $\langle M_2 \rangle$, a entrada para a linha M_3 e a coluna $\langle M_2 \rangle$ é *rejeite*, porque H rejeita a entrada $\langle M_3, \langle M_2 \rangle \rangle$.

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	...
M_1	aceite		aceite		
M_2	aceite	aceite	aceite	aceite	
M_3					...
M_4	aceite	aceite			
$\vdots$			$\vdots$		

FIGURA 4.19

A entrada i, j é *aceite* se M_i aceita $\langle M_j \rangle$.

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	...
M_1	aceite	rejeite	aceite	rejeite	
M_2	aceite	aceite	aceite	aceite	
M_3	rejeite	rejeite	rejeite	rejeite	...
M_4	aceite	aceite	rejeite	rejeite	
$\vdots$			$\vdots$		

FIGURA 4.20

A entrada i, j é o valor de H sobre a entrada $\langle M_i, \langle M_j \rangle \rangle$.

Na figura a seguir, adicionamos D à Figura 4.20. Por nossa hipótese, H é uma MT e o mesmo acontece com D . Conseqüentemente, essa última tem que ocorrer na lista $M_1, M_2, \dots$ de todas as MTs. Note que D computa o oposto das entradas na diagonal. A contradição ocorre no ponto de interrogação, onde a entrada tem de ser o oposto de si mesma.

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	...	$\langle D \rangle$	...
M_1	<u>aceite</u>	rejeite	aceite	rejeite		aceite	
M_2	aceite	<u>aceite</u>	aceite	aceite		aceite	
M_3	rejeite	rejeite	<u>rejeite</u>	rejeite	...	rejeite	...
M_4	aceite	aceite	rejeite	<u>rejeite</u>		aceite	
$\vdots$			$\vdots$		$\ddots$		
D	rejeite	rejeite	aceite	aceite		<u>?</u>	
$\vdots$			$\vdots$				$\ddots$

FIGURA 4.21

Se D estiver na figura, uma contradição ocorre em “?”.

190 CAPÍTULO 4 / DECIDIBILIDADE

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	...
M_1	aceite		aceite		
M_2	aceite	aceite	aceite	aceite	
M_3					...
M_4	aceite	aceite			
$\vdots$			$\vdots$		

FIGURA 4.19

A entrada i, j é *aceite* se M_i aceita $\langle M_j \rangle$.

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	...
M_1	aceite	rejeite	aceite	rejeite	
M_2	aceite	aceite	aceite	aceite	
M_3	rejeite	rejeite	rejeite	rejeite	...
M_4	aceite	aceite	rejeite	rejeite	
$\vdots$			$\vdots$		

FIGURA 4.20

A entrada i, j é o valor de H sobre a entrada $\langle M_i, \langle M_j \rangle \rangle$.

Na figura a seguir, adicionamos D à Figura 4.20. Por nossa hipótese, H é uma MT e o mesmo acontece com D . Conseqüentemente, essa última tem que ocorrer na lista $M_1, M_2, \dots$ de todas as MTs. Note que D computa o oposto das entradas na diagonal. A contradição ocorre no ponto de interrogação, onde a entrada tem de ser o oposto de si mesma.

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	...	$\langle D \rangle$	...
M_1	<u>aceite</u>	rejeite	aceite	rejeite		aceite	
M_2	aceite	<u>aceite</u>	aceite	aceite		aceite	
M_3	rejeite	rejeite	<u>rejeite</u>	rejeite	...	rejeite	...
M_4	aceite	aceite	rejeite	<u>rejeite</u>		aceite	
$\vdots$			$\vdots$		$\ddots$		
D	rejeite	rejeite	aceite	aceite		<u>?</u>	
$\vdots$			$\vdots$				$\ddots$

FIGURA 4.21

Se D estiver na figura, uma contradição ocorre em "?".

UMA LINGUAGEM TURING-IRRECONHECÍVEL

Na seção precedente, exibimos uma linguagem — a saber, A_{MT} — que é indecidível. Agora exibimos uma linguagem que não é sequer Turing-reconhecível. Note que A_{MT} não servirá para esse propósito, porque mostramos que A_{MT} é Turing-reconhecível (página 182). O teorema a seguir mostra que, se uma linguagem e seu complemento forem ambos Turing-reconhecíveis, a linguagem é decidível. Logo, para qualquer linguagem indecidível, ou ela ou seu complemento não é Turing-reconhecível. Lembre-se de que o complemento de uma linguagem é a linguagem constituída de todas as cadeias que não estão nela. Dizemos que uma linguagem é *co-Turing-reconhecível* se ela for o complemento de uma linguagem Turing-reconhecível.

TEOREMA 4.22

Uma linguagem é decidível sse ela é Turing-reconhecível e co-Turing-reconhecível.

Em outras palavras, uma linguagem é decidível exatamente quando ela e seu complemento são ambas Turing-reconhecíveis.

PROVA Temos duas direções a provar. Primeiro, se A for decidível, podemos facilmente ver que tanto A quanto seu complemento $\bar{A}$ são Turing-reconhecíveis. Qualquer linguagem decidível é Turing-reconhecível, e o complemento de uma linguagem decidível também é decidível.

Para a outra direção, se tanto A quanto $\bar{A}$ são Turing-reconhecíveis, fazemos M_1 ser o reconhecedor para A e M_2 o reconhecedor para $\bar{A}$. A máquina de Turing M a seguir é um decisor para A .

$M =$ “Sobre a entrada w :

1. Rode ambas, M_1 e M_2 , sobre a entrada w em paralelo.
2. Se M_1 aceita, *aceite*; se M_2 aceita, *rejeite*.”

Rodar as duas máquinas em paralelo significa que M tem duas fitas, uma para simular M_1 e a outra para simular M_2 . Nesse caso, M alternativamente simula um passo de cada máquina, o que continua até que uma delas aceite.

Agora, mostramos que M decide A . Toda cadeia w ou está em A ou em $\bar{A}$. Consequentemente, ou M_1 ou M_2 tem de aceitar w . Uma vez que M pára sempre que M_1 ou M_2 aceita, M sempre pára e, portanto, é um decisor. Além disso, ela aceita todas as cadeias em A e rejeita todas as cadeias que não estão em A . Logo, M é um decisor para A e, consequentemente, A é decidível.

COROLÁRIO 4.23

$\bar{A}_{MT}$ não é Turing-reconhecível.

PROVA Sabemos que A_{MT} é Turing-reconhecível. Se $\overline{A_{MT}}$ também fosse Turing-reconhecível, A_{MT} seria decidível. O Teorema 4.11 nos diz que A_{MT} não é decidível, portanto $\overline{A_{MT}}$ não pode ser Turing-reconhecível.

EXERCÍCIOS

^R4.1 Responda a cada um dos itens abaixo para o AFD M e dê razões para suas respostas.



- | | |
|---|---|
| a. $\langle M, 0100 \rangle \in A_{AFD}?$ | d. $\langle M, 0100 \rangle \in A_{EXR}?$ |
| b. $\langle M, 011 \rangle \in A_{AFD}?$ | e. $\langle M \rangle \in V_{AFD}?$ |
| c. $\langle M \rangle \in A_{AFD}?$ | f. $\langle M, M \rangle \in EQ_{AFD}?$ |

4.2 Considere o problema de se determinar se um AFD e uma expressão regular são equivalentes. Expresse esse problema como uma linguagem e mostre que ele é decidível.

4.3 Seja $TODAS_{AFD} = \{\langle A \rangle \mid A \text{ é um AFD e } L(A) = \Sigma^*\}$. Mostre que $TODAS_{AFD}$ é decidível.

4.4 Seja $A\epsilon_{GLC} = \{\langle G \rangle \mid G \text{ é uma GLC que gera } \epsilon\}$. Mostre que $A\epsilon_{GLC}$ é decidível.

4.5 Seja X o conjunto $\{1, 2, 3, 4, 5\}$ e Y o conjunto $\{6, 7, 8, 9, 10\}$. Descrevemos as funções $f: X \rightarrow Y$ e $g: X \rightarrow Y$ nas tabelas abaixo. Responda a cada item e dê uma razão para cada resposta negativa.

n	$f(n)$
1	6
2	7
3	6
4	7
5	6

n	$g(n)$
1	10
2	9
3	8
4	7
5	6

- ^Ra. f é um-para-um? ^Rd. g é um-para-um?
b. f é sobrejetora? e. g é sobrejetora?
c. f é uma correspondência? f. g é uma correspondência?

- 4.6 Seja B o conjunto de todas as seqüências infinitas sobre $\{0,1\}$. Mostre que B é incontável, usando uma prova por diagonalização.
4.7 Seja $T = \{(i, j, k) \mid i, j, k \in \mathcal{N}\}$. Mostre que T é contável.
4.8 Revise a maneira pela qual definimos conjuntos como sendo do mesmo tamanho na Definição 4.12 (página 184). Mostre que “é do mesmo tamanho” é uma relação de equivalência.

PROBLEMAS

- ^R4.9 Seja $INFINITA_{AFD} = \{\langle A \rangle \mid A \text{ é um AFD e } L(A) \text{ é uma linguagem infinita}\}$. Mostre que $INFINITA_{AFD}$ é decidível.
4.10 Seja $INFINITA_{AP} = \{\langle M \rangle \mid M \text{ é um AP e } L(M) \text{ é uma linguagem infinita}\}$. Mostre que $INFINITA_{AP}$ é decidível.
^R4.11 Seja $A = \{\langle M \rangle \mid M \text{ é um AFD que não aceita nenhuma cadeia contendo um número ímpar de 1s}\}$. Mostre que A é decidível.
4.12 Seja $A = \{\langle R, S \rangle \mid R \text{ e } S \text{ são expressões regulares e } L(R) \subseteq L(S)\}$. Mostre que A é decidível.
^R4.13 Seja $\Sigma = \{0,1\}$. Mostre que o problema de se determinar se uma GLC gera alguma cadeia em 1^* é decidível. Em outras palavras, mostre que

$$\{(G) \mid G \text{ é uma GLC sobre } \{0,1\} \text{ e } 1^* \cap L(G) \neq \emptyset\}$$
é uma linguagem decidível.
4.14 Mostre que o problema de se determinar se uma GLC gera todas as cadeias em 1^ é decidível. Em outras palavras, mostre que $\{(G) \mid G \text{ é uma GLC sobre } \{0,1\} \text{ e } 1^* \subseteq L(G)\}$ é uma linguagem decidível.
4.15 Seja $A = \{\langle R \rangle \mid R \text{ é uma expressão regular que descreve uma linguagem contendo pelo menos uma cadeia } w \text{ que tem 111 como uma subcadeia (isto é, } w = x111y \text{ para alguma } x \text{ e alguma } y)\}$. Mostre que A é decidível.
4.16 Prove que EQ_{AFD} é decidível testando os dois AFDs sobre todas as cadeias até um certo tamanho. Calcule um tamanho que funcione.
*4.17 Seja C uma linguagem. Prove que C é Turing-reconhecível sse existe uma linguagem decidível D tal que $C = \{x \mid \exists y ((x, y) \in D)\}$.
4.18 Sejam A e B duas linguagens disjuntas. Digamos que a linguagem C *separa* A e B se $A \subseteq C$ e $B \subseteq \bar{C}$. Mostre que quaisquer duas linguagens co-Turing-reconhecíveis disjuntas são separáveis por alguma linguagem decidível.
4.19 Seja $S = \{\langle M \rangle \mid M \text{ é um AFD que aceita } w^R \text{ sempre que ele aceita } w\}$. Mostre que S é decidível.

- 4.20 Uma linguagem é *livre-de-prefixo* se nenhum membro é um prefixo próprio de um outro membro. Seja $LIVRE-DE-PREFIXO_{EXR} = \{R \mid R \text{ é uma expressão regular onde } L(R) \text{ é livre-de-prefixo}\}$. Mostre que $LIVRE-DE-PREFIXO_{EXR}$ é decidível. Por que uma abordagem similar falha em mostrar que $LIVRE-DE-PREFIXO_{GLC}$ é decidível?
- R* 4.21 Digamos que um AFN é *ambíguo* se ele aceita alguma cadeia ao longo de dois ramos diferentes da computação. Seja $AMBIG_{AFN} = \{\langle N \rangle \mid N \text{ é um AFN ambíguo}\}$. Mostre que $AMBIG_{AFN}$ é decidível. (Sugestão: Uma maneira elegante de resolver este problema é construir um AFD apropriado e aí então rodar E_{AFD} sobre ele.)
- 4.22 Um *estado inútil* em um autômato com pilha nunca é atingido sobre qualquer cadeia de entrada. Considere o problema de se determinar se um autômato com pilha tem quaisquer estados inúteis. Formule esse problema como uma linguagem e mostre que ele é decidível.
- R* 4.23 Seja $BAL_{AFD} = \{\langle M \rangle \mid M \text{ é um AFD que aceita alguma cadeia contendo igual número de 0s e 1s}\}$. Mostre que BAL_{AFD} é decidível. (Dica: Os teoremas sobre LLCs são úteis aqui.)
- * 4.24 Seja $PAL_{AFD} = \{\langle M \rangle \mid M \text{ é um AFD que aceita algum palíndromo}\}$. Mostre que PAL_{AFD} é decidível. (Dica: Os teoremas sobre LLCs são úteis aqui.)
- * 4.25 Seja $E = \{\langle M \rangle \mid M \text{ é um AFD que aceita alguma cadeia com mais 1s que 0s}\}$. Mostre que E é decidível. (Dica: Os teoremas sobre LLCs são úteis aqui.)
- 4.26 Seja $C = \{\langle G, x \rangle \mid G \text{ é uma GLC que gera alguma cadeia } w, \text{ onde } x \text{ é uma subcadeia de } w\}$. Mostre que C é decidível. (Sugestão: Uma solução elegante para esse problema usa o decisor para V_{GLC} .)
- 4.27 Seja $C_{GLC} = \{\langle G, k \rangle \mid L(G) \text{ contém exatamente } k \text{ cadeias, onde } k \geq 0, \text{ ou então } k = \infty\}$. Mostre que C_{GLC} é decidível.
- 4.28 Seja A uma linguagem Turing-reconhecível consistindo de descrições de máquinas de Turing, $\{\langle M_1 \rangle, \langle M_2 \rangle, \dots\}$, onde toda M_i é um decisor. Prove que alguma linguagem decidível D não é decidida por nenhum decisor M_i cuja descrição aparece em A . (Dica: Você pode achar útil considerar um enumerador para A .)

SOLUÇÕES SELECIONADAS

- 4.1 (a) Sim. O AFD M aceita 0100.
 (b) Não. M não aceita 011.
 (c) Não. Essa entrada tem apenas um único componente e, portanto, não é da forma correta.
 (d) Não. O primeiro componente não é uma expressão regular e por isso a entrada não é da forma correta.
 (e) Não. A linguagem de M não é vazia.
 (f) Sim. M aceita a mesma linguagem que si própria.
- 4.5 (a) Não, f não é um-para-um porque $f(1) = f(3)$.
 (d) Sim, g é um-para-um.

4.9 A seguinte MT I decide $INFINITA_{AFD}$.

$I =$ "Sobre a entrada $\langle A \rangle$ onde A é um AFD:

1. Seja k o número de estados de A .
2. Construa um AFD D que aceite todas as cadeias de comprimento k ou mais.
3. Construa um AFD M tal que $L(M) = L(A) \cap L(D)$.
4. Teste $L(M) = \emptyset$, usando o decisor T de V_{AFD} do Teorema 4.4.
5. Se T aceita, *rejeite*; se T rejeita, *aceite*."

Esse algoritmo funciona porque um AFD que aceite uma quantidade infinita de cadeias tem que aceitar cadeias arbitrariamente longas. Por conseguinte, esse algoritmo aceita tais AFDs. Reciprocamente, se o algoritmo aceita um AFD, o AFD aceita alguma cadeia de comprimento k ou mais, onde k é o número de estados do AFD. Essa cadeia pode ser bombeada da maneira prescrita pelo lema do bombeamento para linguagens regulares para se obter uma quantidade infinita de cadeias aceitas.

4.11 A seguinte MT decide A .

"Sobre a entrada $\langle M \rangle$:

1. Construa um AFD O que aceite toda cadeia contendo um número ímpar de 1s.
2. Construa o AFD B tal que $L(B) = L(M) \cap L(O)$.
3. Teste se $L(B) = \emptyset$, usando o decisor T de V_{AFD} do Teorema 4.4.
4. Se T aceita, *aceite*; se T rejeita, *rejeite*."

4.13 Você mostrou no Problema 2.18 que, se C for uma linguagem livre-do-contexto e R uma linguagem regular, então $C \cap R$ é livre-do-contexto. Consequentemente, $1^* \cap L(G)$ é livre-do-contexto. A seguinte MT decide A .

"Sobre a entrada $\langle G \rangle$:

1. Construa a GLC H tal que $L(H) = 1^* \cap L(G)$.
2. Teste se $L(H) = \emptyset$, usando o decisor R de V_{GLC} do Teorema 4.8.
3. Se R aceita, *rejeite*; se R rejeita, *aceite*."

4.21 O seguinte procedimento decide $AMBIG_{AFN}$. Dado um AFN N , construímos um AFD D que simula N e aceita uma cadeia sse ela for aceita por N ao longo de dois ramos de computação diferentes. Aí, então, usamos um decisor para V_{AFD} para determinar se D aceita quaisquer cadeias.

Nossa estratégia para construir D é similar à da conversão de AFN para AFD na prova do Teorema 1.39. Simulamos N mantendo uma pedra sobre cada estado ativo. Começamos colocando uma pedra vermelha sobre o estado inicial e sobre cada estado atingível a partir do inicial ao longo de transições ϵ . Movemos, adicionamos e removemos pedras de acordo com as transições de N , preservando a cor das pedras. Sempre que duas ou mais pedras são movidas para o mesmo estado, substituímos suas pedras por uma pedra azul. Após ler a entrada, aceitamos se uma pedra azul estiver sobre um estado de aceitação de N .

O AFD D tem um estado correspondente a cada posição possível de pedras. Para cada estado de N , três possibilidades ocorrem: ele pode conter uma pedra vermelha, uma pedra azul ou nenhuma pedra. Por conseguinte, se N tiver n estados,